# 01 - Azure Virtual Machine

05 April 2026 08:02 PM

## 1. Introduction to Virtualization

### What is Virtualization?

**Virtualization** is the process of creating **multiple virtual computers** from a **single physical server**.

### Example

1 Physical Server → Multiple Virtual Machines

```
Physical Server
 ├── VM 1 (Windows)
 ├── VM 2 (Linux)
 └── VM 3 (Ubuntu)
```

### Why Virtualization?

- Save cost 💰
- Better resource usage
- Easy to manage
- High scalability
- Isolation between applications

## 2. Virtual Machine (VM)

### What is Virtual Machine?

A **Virtual Machine** is a **software-based computer** running on a physical machine.

You can:

- Install OS
- Install software
- Deploy application
- Use like your own computer

### Example

- Windows VM
- Linux VM
- Ubuntu VM

## 3. Hypervisor

### What is Hypervisor?

A **Hypervisor** is software that **creates and manages Virtual Machines**

### Types of Hypervisors

### Type 1 (Bare Metal)

Runs directly on hardware

Examples:

- Hyper-V
- VMware ESXi

### Type 2 (Hosted)

Runs on OS

Examples:

- VirtualBox

- VMware Workstation

### 4. Azure Virtual Machine

Azure Virtual Machine is a **cloud-based computer**.

You can:

- Create server
- Deploy applications
- Install software
- Manage networking

#### Features of Azure VM

- Multiple OS support (Windows / Linux)
- Full control
- Pay as you use
- High scalability
- Custom configuration
- Secure networking

#### Use Cases

- Hosting website
- Running backend services
- Testing environments
- Development server
- Production server

### 5. Virtual Machine Series and Sizes

Azure provides different VM sizes:

Based on:

- CPU
- RAM
- Storage
- Performance

#### Types

#### General Purpose

Balanced CPU and RAM

Example:

- Web servers
- Development environments

#### Compute Optimized

More CPU

Example:

- High traffic apps

#### Memory Optimized

More RAM

Example:

- Databases

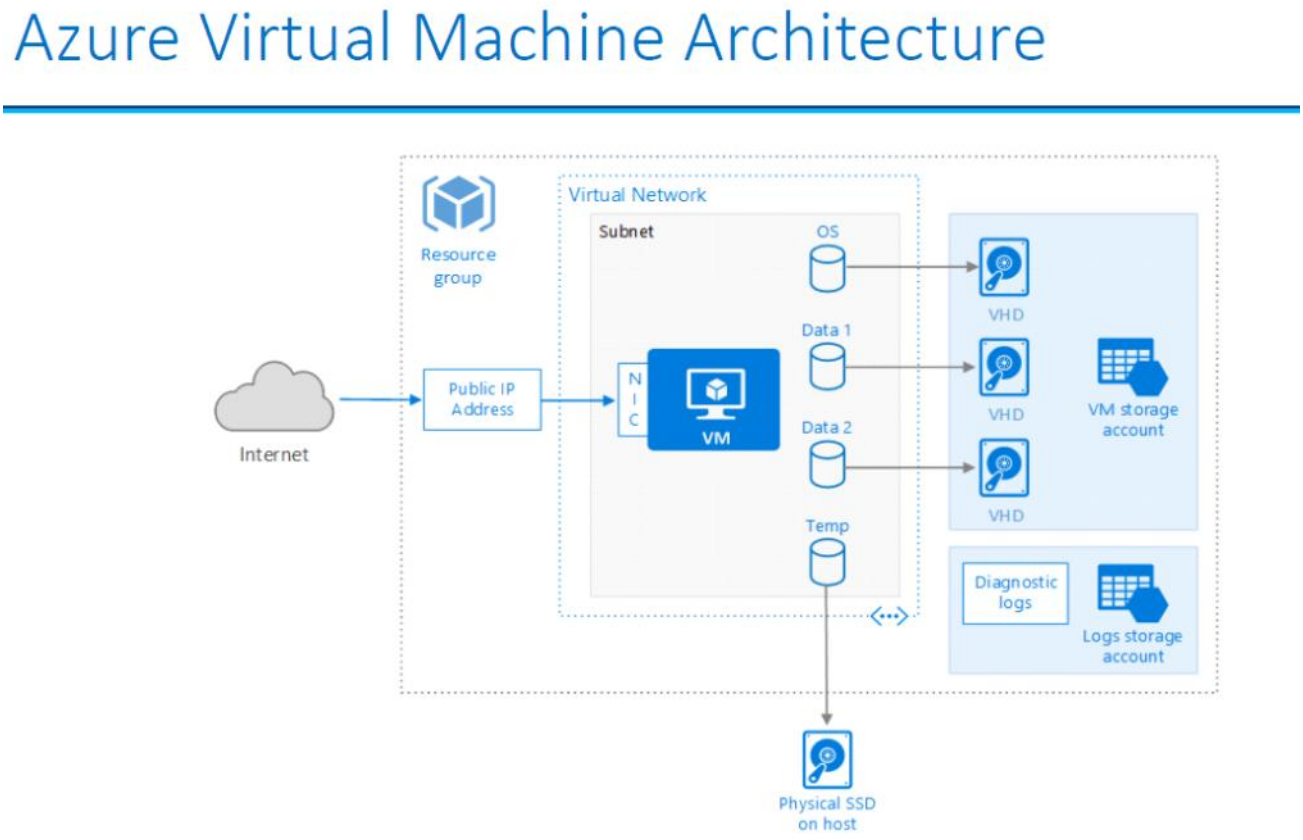
#### Storage Optimized

More disk speed

Example:

- Big data

### 6. Azure Virtual Machine Architecture




Important Components:

### 6.1 Resource Group

A **Resource Group** is a container that holds Azure resources.
Example:

- VM
- Storage
- Network

### 6.2 Virtual Machine

Your actual server
Example:

- Windows Server VM
- Linux VM

### 6.3 OS Disk

Disk where OS is installed
Example:

- Windows OS disk

Important:

- Persistent
- Not deleted after restart

### 6.4 Temporary Disk

Temporary storage

Used for:

- Cache
- Temp files

Important:

- Deleted after restart

### 6.5 Data Disk

Used for application data
Example:

- Database
- Files
- Images

Important:

- Persistent storage

## 7. Networking Components

### 7.1 Virtual Network (VNet)

Private network in Azure
Example:

- Internal network for VMs

### 7.2 Subnet

Small network inside VNet
Example:

- Frontend subnet
- Backend subnet

### 7.3 Public IP

Used to access VM from internet
Example:

- RDP access

### 7.4 Private IP

Used for internal communication
Example:

- VM to database communication

### 7.5 Network Interface (NIC)

Connects VM to network
Example:

- Ethernet card

## 8. Network Security Group (NSG)

Controls inbound and outbound traffic
Example Rules:

| Port | Use |
|---|---|

80 HTTP

443 HTTPS

3389 RDP

22 SSH

### 9. Load Balancer

Distributes traffic across multiple VMs
Example:

User
↓
Load Balancer
↓
VM1 VM2 VM3
Benefits:

- High availability
- Better performance

### 10. Diagnostics

Used for:

- Logging
- Monitoring
- Troubleshooting

Example:

- CPU usage
- Memory usage
- Errors

### 11. Accelerated Networking

Improves network performance
Benefits:

- Low latency
- High throughput

### 12. Web Deployment

Steps to Deploy .NET App

#### Step 1: Install IIS

```
Install-WindowsFeature -name Web-Server –IncludeManagementTools
```

#### Step 2: Install ASP.NET

```
Install-WindowsFeature Web-Asp-Net45
```

#### Step 3: Install .NET Core Hosting Bundle

Download and install

## 13. Web Deployment Using Visual Studio

Steps:

1. Install Web Management Service
2. Install Web Deploy
3. Open Port 8172
4. Deploy application

## 14. Opening Ports in Azure VM

Steps:

1. Add inbound rule in NSG
2. Add firewall rule
3. Access from internet

Example:

- Port 80 → Website
- Port 3389 → RDP

## 15. Data Sharing Between VMs

Using Azure File Service

Example:

- VM1 share files to VM2

Benefits:

- Easy file sharing
- Centralized storage

## 16. Virtual Machine Scaling

Scaling means increasing or decreasing resources

### 16.1 Vertical Scaling

Increase VM size

Example:

- Increase CPU
- Increase RAM

Scale Up / Scale Down

### 16.2 Horizontal Scaling

Increase number of VMs

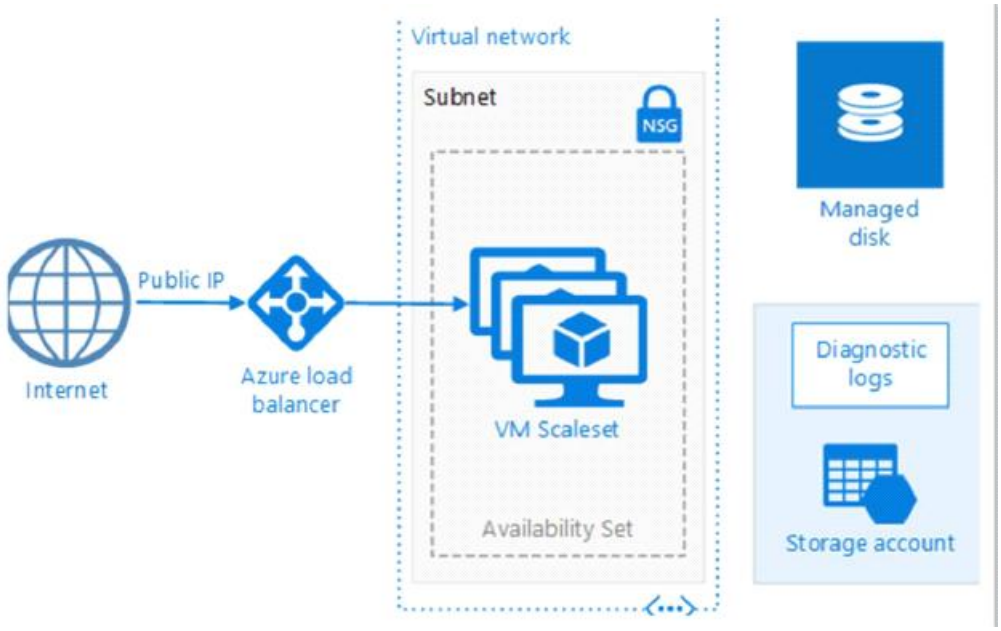
Example:

1 VM → 3 VM

Used when:

CPU > 70%

## 17. Virtual Machine Scale Sets




Used for:

- Auto scaling
- High availability
- Large scale apps

Features:

- Auto scaling
- Load balancing
- High availability

Example:

- Web servers
- Microservices

### 18. Custom Domain

Instead of:
20.10.10.1
Use:
www.mywebsite.com

### 19. Configure DNS

Steps:

1. Create domain
2. Map IP
3. Configure DNS

### 20. Important Interview Questions

**What is Azure VM?**
Cloud-based virtual server
**What is VNet?**
Private network in Azure
**What is NSG?**
Firewall for VM
**OS Disk vs Data Disk?**

| OS Disk | Data Disk |
|---|---|

| | |
|---|---|
| OS installed | Data stored |
| Persistent | Persistent |

## 21. Real-world Example

Deploy ASP.NET Application

Steps:

1. Create VM
2. Install IIS
3. Install .NET
4. Deploy app
5. Open port 80
6. Access website

## 22. Summary

Azure Virtual Machine allows you to:

- Create server
- Deploy applications
- Manage networking
- Scale applications
- Monitor performance

# 02 - Azure Storage

05 April 2026 08:22 PM

## 1. What is Azure Storage Account?

A **Storage Account** in Microsoft Azure is a **container that holds all Azure storage services**.

It is similar to:

- Google Drive (store files)
- Database (store structured data)
- File Server (store shared files)

**Simple Definition**

Azure Storage Account is a **central place to store data in Azure Cloud**.

## 2. Why Azure Storage Account?

Azure Storage Account is used to store:

- Files
- Images
- Videos
- Logs
- Backups
- Database-like data

## 3. Types of Azure Storage Services

Azure Storage Account provides **4 main storage services**:

| Storage Type | Used For | Example |
|---|---|---|
| Blob Storage | Files & media | Images, Videos |
| File Storage | Shared file system | Network drive |
| Queue Storage | Messaging | Background jobs |
| Table Storage | NoSQL database | Key-value data |

## 4. Azure Storage Types Explained

### 4.1 Blob Storage (Most Important)

Blob = Binary Large Object

Used for:

- Images
- Videos
- Documents
- Backup files
- Logs

Example:

User uploads profile photo → Stored in Blob Storage

Blob Storage Structure:

Storage Account
└── Container
└── Blob (files)
Example:

mystorageaccount
└── images
└── photo.jpg
Types of Blob:

| Blob Type | Description |
|---|---|
| Block Blob | Images, documents |
| Append Blob | Logging |
| Page Blob | Virtual Machines |

### 4.2 File Storage
Used to store **shared files**
Example:
- Shared folder
- Network drive

Example:

Company shared folder → Azure File Storage
Access using:
- SMB Protocol
- NFS Protocol

### 4.3 Queue Storage
Used for **message queue system**
Example:

User uploads file → Message goes to queue → Background service processes file
Use case:
- Background processing
- Order processing
- Email sending

### 4.4 Table Storage
Used for **NoSQL structured data**
Example:

| UserId | Name | City |
|---|---|---|
| 1 | John | Pune |
| 2 | Sam | Mumbai |

Features:
- Fast
- Cheap

- Scalable

### 5. Storage Account Types

Azure provides different storage account types:

| Type | Description |
|---|---|
| General Purpose v2 | Most used |
| General Purpose v1 | Old version |
| Blob Storage | Only blob |
| BlockBlobStorage | High performance |
| FileStorage | Premium file storage |

Recommended:
**General Purpose v2 (GPv2)**

### 6. Storage Performance Types

Azure provides two performance types:

**Standard**

- HDD based
- Low cost
- Normal performance

**Premium**

- SSD based
- High performance
- Expensive

### 7. Storage Access Tiers (Blob Storage)

Azure provides 3 access tiers:

| Tier | Use case |
|---|---|
| Hot | Frequently used data |
| Cool | Rarely used data |
| Archive | Rarely used long-term |

Example:

Recent files → Hot
Old files → Cool
Backup → Archive

### 8. Replication Options (Very Important)

Azure provides data redundancy:

| Replication | Description |
|---|---|
| LRS | Local redundancy |
| ZRS | Zone redundancy |
| GRS | Geo redundancy |
| RA-GRS | Read access geo |

Explanation:

### LRS

- 3 copies in same datacenter

### ZRS

- Copies in different zones

### GRS

- Copies in different region

Example:

India → Singapore backup

### 9. Storage Security

Azure Storage Security Options:

### 1. Access Keys

Two keys:

- Key1
- Key2

Used for:

- Authentication

### 2. SAS Token (Important)

SAS = Shared Access Signature

Used for:

- Temporary access

Example:

User access only 1 hour

### 3. Azure AD Authentication

Best practice:

Use:

- Azure AD
- RBAC roles

### 10. Storage Networking

Azure Storage supports:

- Public access
- Private endpoint
- Firewall

Example:

Allow only company IP

### 11. Storage Account Naming Rules

Rules:

- Must be unique
- Lowercase only

- 3–24 characters
- No special characters

Example:

mycompanydata

### 12. How to Create Azure Storage Account (Step-by-Step)

Step 1:
Go to Azure Portal
Step 2:
Create Resource
Step 3:
Search "Storage Account"
Step 4:
Fill details:

- Subscription
- Resource group
- Storage name
- Region
- Performance

Step 5:
Create

### 13. Real-World Example

E-commerce Website:

| Feature | Storage |
|---|---|
| Product images | Blob |
| Logs | Append blob |
| Orders queue | Queue |
| User data | Table |

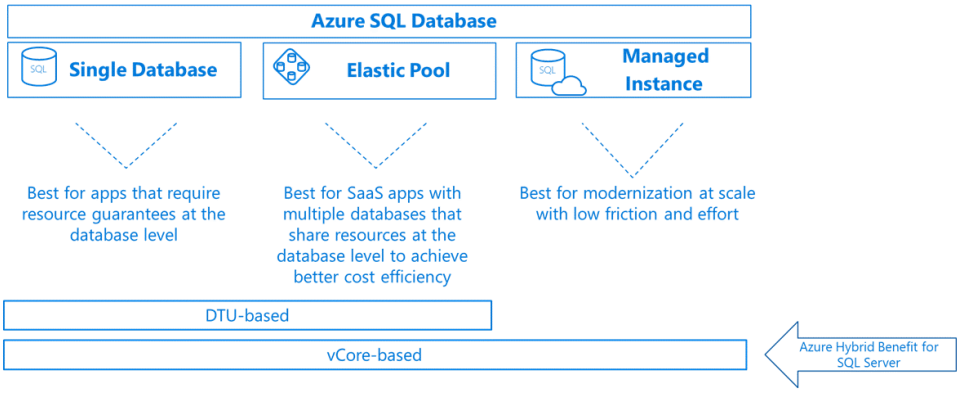
# 03 - Azure SQL

07 April 2026 11:30 PM

## 1. What is Azure SQL?

**Azure SQL** is a **cloud-based relational database service** provided by Microsoft on
Microsoft Azure
Simple definition:

Azure SQL is a **managed SQL Server database in the cloud**




## 2. Why Azure SQL?

Instead of installing:

- SQL Server
- Hardware
- Backup systems
- Maintenance

Azure SQL **handles everything automatically**:

☑ Backup
☑ Security
☑ Scaling
☑ High Availability
☑ Patching

## 3. Types of Azure SQL Services (Very Important)

Azure SQL has **3 main types**:

| Type | Description | Use Case |
|---|---|---|
| Azure SQL Database | Single database | Small apps |
| Azure SQL Managed Instance | Full SQL server | Enterprise apps |
| SQL Server on Azure VM | Full control | Legacy apps |

**Database Deployment Options**

Azure SQL provides:

**Platform Managed**

- Azure SQL Database
- Managed Instance

**Self Managed**

- SQL Server on Azure VM

- **Azure SQL Database**: A Platform-as-a-Service (PaaS) offering for single databases or elastic pools.
- **Azure SQL Managed Instance**: A managed SQL Server instance with near 100% compatibility with on-premises SQL Server.
- **SQL Server on Azure VM**: Infrastructure-as-a-Service (IaaS) option for full control over SQL Server in a VM.

**4. Azure SQL Database (Most Used)**
This is **PaaS (Platform as a Service)**
Features:

- Automatic backup
- Auto scaling
- High availability
- Managed infrastructure

Example:

Web Application → Azure SQL Database
Best for:

- Web applications
- APIs
- Mobile apps

**5. Azure SQL Managed Instance**
This is **near full SQL Server in cloud**
Used when:

- Migrating on-prem SQL Server
- Need advanced features

Supports:

- SQL Agent
- Linked Server
- Cross database query

Example:

On-Prem SQL Server → Azure SQL Managed Instance

**6. SQL Server on Azure VM**
This is **IaaS (Infrastructure as a Service)**
You manage:

- OS
- SQL Server
- Backup
- Security

Used when:

- Full control required
- Legacy applications

Example:

Install SQL Server on Virtual Machine

**7. Azure SQL Architecture**

Application
↓
Azure SQL Server
↓
Azure SQL Database

- **Connection Strings**: Use secure authentication (Azure AD, Managed Identity).
- **Pricing Models**:
- **DTU-based**: Fixed compute units.
- **vCore-based**: Flexible compute and storage scaling.
- **Backup & Restore**: Automated backups with configurable retention (7–35 days).
- **Geo-Replication**: Active geo-replication for disaster recovery.

**8. Azure SQL Server vs Azure SQL Database**

| Azure SQL Server | Azure SQL Database |
|---|---|
| Logical server | Actual database |
| Multiple DBs | Single database |
| Security config | Data storage |

**Comparison Table**

| Aspect | SQL Server | SQL Database |
|---|---|---|
| **Definition** | Software/engine that manages databases | A structured collection of data |
| **Scope** | Can host multiple databases | Single database instance |
| **Role in Azure** | Logical container/public endpoint | Individual database you create |
| **Management** | Provides security, backups, performance tuning | Stores and organizes data |
| **Use Case** | Needed to run and manage databases | Used to store application data |

- Example:
- You create an **Azure SQL Server** named myserver.database.windows.net.
- Inside it, you create **databases** like SalesDB, HRDB, InventoryDB.

- Applications connect to myserver.database.windows.net and specify which database to use.

**9. Azure SQL Performance Tiers**
Azure SQL provides **3 performance tiers**
**1. Basic**
- Low cost
- Small apps
- Development

**2. Standard**
- Medium performance
- Production apps

**3. Premium**
- High performance
- Enterprise apps

**10. Compute Tiers**
Azure SQL provides:
**Provisioned Compute**
- Fixed CPU
- Fixed memory

**Serverless Compute**
- Auto scaling
- Pay per use

Best for:
- Intermittent usage

**11. Azure SQL Service Tiers**

| Tier | Description |
|---|---|
| General Purpose | Balanced |
| Business Critical | High performance |
| Hyperscale | Very large database |

**12. Azure SQL Security**
Azure SQL provides:
**1. Firewall**
Allow specific IP
Example:

Allow only office IP

**2. Authentication Types**
- SQL Authentication
- Azure AD Authentication

Best practice:
Use Azure AD authentication

**3. Encryption**
Azure SQL provides:
- Transparent Data Encryption (TDE)
- Always Encrypted

**13. Azure SQL Backup**
Azure SQL automatically:
- Daily backup
- Weekly backup
- Transaction log backup

Retention:
- 7 to 35 days

Long-term retention also available

**14. High Availability**
Azure SQL automatically provides:
- 99.99% availability
- Auto failover

No manual configuration required

**15. Azure SQL Scaling**
Azure SQL supports:
**Vertical Scaling**
Increase:
- CPU
- Memory

**Horizontal Scaling**
- Read replicas
- Geo replication

**16. Geo Replication**
Azure SQL supports:
- Primary region
- Secondary region

Example:

India → Singapore backup

**17. Azure SQL Pricing Factors**
Azure SQL pricing based on:
- Storage
- Compute
- Tier
- Region

### 18. Azure SQL vs SQL Server

| SQL Server | Azure SQL |
|---|---|
| On-premise | Cloud |
| Manual backup | Automatic backup |
| Manual scaling | Auto scaling |
| Hardware required | No hardware |

### 19. How to Create Azure SQL Database

Step-by-Step:
Step 1:
Go to Azure Portal
Step 2:
Create Resource
Step 3:
Select SQL Database
Step 4:
Fill details:

- Database name
- Server name
- Region
- Pricing tier

Step 5:
Create

### 20. Real World Example

E-commerce Application:

| Component | Use |
|---|---|
| Product Data | Azure SQL |
| User Data | Azure SQL |
| Orders | Azure SQL |

### 21. Azure SQL Components

Azure SQL includes:

- Logical Server
- Database
- Tables
- Views
- Stored Procedures

### 22. Azure SQL Features

Important Features:
☑ Automatic backup
☑ High availability
☑ Auto scaling

☑ Security
☑ Monitoring

**23. Monitoring Azure SQL**
Azure provides:
- Metrics
- Logs
- Alerts

Use:
- Azure Monitor
- Query performance insights

**24. Azure SQL Connectivity**
Connect using:
- SQL Server Management Studio (SSMS)
- Azure Data Studio
- Application connection string

**25. Connection String Example**

Server=tcp:server.database.windows.net
Database=mydb
User ID=admin
Password=****

Administration & Tools
- **Firewall & Networking**: Configure firewall rules, private endpoints, and VNET integration.
- **Monitoring**: Use **Azure Monitor**, **SQL Insights**, and **Database Watcher**.
- **Automation**: ARM templates, PowerShell, and CLI for deployment and scaling.
- **Migration**: Data Migration Assistant (DMA) and Azure Database Migration Service (DMS).

---

**What are SQL Elastic Pools?**
**Azure SQL Elastic Pools** in Microsoft Azure SQL Database allow **multiple databases to share resources** (CPU, Memory, Storage) instead of assigning dedicated resources to each database.
This helps:
- Reduce cost 💰
- Improve performance ⚡
- Simplify management 🛠️
- Handle varying workloads 📈

**Why Elastic Pools?**
Normally:
Each database has **dedicated resources**
Problem:

- Some databases use **very low resources**
- Some databases use **high resources**
- Resources get **wasted**

Elastic Pools solve this problem by **sharing resources across databases**

**Example Scenario**

You have 10 databases:

| Database | Usage |
|---|---|
| DB1 | High |
| DB2 | Low |
| DB3 | Low |
| DB4 | Medium |
| DB5 | Low |

Instead of assigning **10 servers**, Elastic Pool uses:

☑ **One shared resource pool**

**How Elastic Pools Work**

Elastic Pool
- ├── Database 1
- ├── Database 2
- ├── Database 3
- ├── Database 4
- └── Database 5

All databases **share CPU, memory, storage**

**Steps to Create Elastic Pool (Azure Portal)**

**Step 1**

Login to Azure Portal

**Step 2**

Go to SQL Databases

**Step 3**

Click "Create Elastic Pool"

**Step 4**

Enter Details
- Pool name
- Server
- Resource size

**Step 5**

Add Databases

**Step 6**

Click Create

**Monitor an Elastic Pool**

Azure provides monitoring tools:

**Metrics Available**

- CPU usage
- Memory usage
- Storage usage
- DTU consumption

**Monitoring Tools**
You can monitor using:
- Azure Portal
- Azure Monitor
- Alerts

**Scaling Elastic Pool**
You can:
- Increase pool size
- Decrease pool size
- Add databases
- Remove databases

Scaling is **online and automatic**

**Common Use Cases**
**SaaS Applications**
Each customer has separate database
**Multi-tenant Applications**
Multiple tenants share infrastructure
**Microservices**
Multiple small databases

---

**Introduction to Managed Instance**
**Azure SQL Managed Instance** is a **fully managed cloud database service** provided by Microsoft on the Microsoft Azure that offers **near 100% compatibility with SQL Server**.
It is designed to **migrate on-premises SQL Server databases to the cloud with minimal changes**.

**Why Azure SQL Managed Instance?**
Azure SQL Managed Instance is mainly used for:
☑ Lift-and-shift migrations
☑ SQL Server compatibility
☑ Reduced maintenance
☑ Automatic updates
☑ Built-in security
☑ High availability

**Azure SQL Managed Instance vs Azure SQL Database**

| **Feature** | **Managed Instance** | **Azure SQL Database** |
|---|---|---|

| | | |
|---|---|---|
| SQL Server compatibility | High | Partial |
| Instance-level features | Supported | Limited |
| Migration effort | Very low | Medium |
| Isolation | Fully isolated | Shared environment |
| Best for | Migration | Cloud-native apps |

**When to Use Azure SQL Managed Instance**

Use Managed Instance when:

☑ Migrating on-prem SQL Server
☑ Using SQL Agent Jobs
☑ Using Linked Servers
☑ Using Cross-database queries
☑ Using CLR

**Key Features of Azure SQL Managed Instance**

**1. Fully Managed Service**

Microsoft manages:

- OS updates
- SQL patches
- Backups
- High availability
- Failover

You only manage:

- Databases
- Queries
- Security

**2. High SQL Server Compatibility**

Supports most SQL Server features:

- SQL Agent
- Linked Servers
- Cross Database Queries
- CLR
- Service Broker

This makes migration **very easy**

**3. Built-in High Availability**

Azure provides:

- Automatic failover
- Multiple replicas
- 99.99% uptime

No manual setup required.

**4. Automatic Backups**

Azure automatically performs:

- Full backup

- Differential backup
- Transaction log backup

Restore Options:

- Point-in-time restore
- Geo restore

**Creating Managed Instance**

You can create Managed Instance using:

- Azure Portal
- Azure CLI
- Azure PowerShell
- ARM Templates

**Steps to Create Managed Instance (Azure Portal)**

Step 1
Go to Azure Portal
Step 2
Search "SQL Managed Instance"
Step 3
Click Create
Step 4
Fill Details

- Instance Name
- Region
- Pricing tier
- Storage

Step 5
Create

**Easily Migrate SQL Server Apps to Cloud**

Azure provides migration tools:

- Azure Database Migration Service
- SQL Server Management Studio
- Backup and Restore

Migration Methods:

1. Backup and Restore
2. Azure Database Migration Service
3. Transactional replication

**Always Operate on Latest Version of SQL**

Azure automatically:

- Updates SQL Server
- Applies patches
- Applies security updates

Benefits:

☑ No downtime planning
☑ No manual upgrades
☑ Always latest features

**Optimize Price Performance and Lower Costs**
Azure Managed Instance helps reduce cost by:
- Pay-as-you-go model
- Reserved instances
- Auto scaling

Cost depends on:
- vCore
- Storage
- Region

**Maintain SQL Server Compatibility**
Managed Instance supports:
- SQL Server Agent
- Database Mail
- Linked Servers
- CLR
- Service Broker

This allows:
- Easy migration
- Minimal code changes

**Increase Database Administrator Productivity**
DBA tasks automated:
- Backup
- Patch updates
- Monitoring
- Scaling

DBA can focus on:
- Performance tuning
- Query optimization
- Security

**Fully Isolated and Secure Environment**
Azure Managed Instance runs inside:
- Virtual Network (VNet)
- Private IP

Benefits:
☑ Network isolation
☑ Secure communication
☑ Private access

**Security Features**
Azure Managed Instance provides:
- Encryption at rest
- Encryption in transit
- Firewall rules

- Azure Active Directory authentication
- Auditing
- Threat detection

**Compliance Features**
Azure supports compliance:
- ISO certification
- GDPR compliance
- SOC compliance

Enterprise-grade security

**Managed Instance Architecture**

Application
↓
Virtual Network (VNet)
↓
Azure SQL Managed Instance
↓
Databases

**Pricing for Managed Instance**
Pricing depends on:
**Compute**
- vCore based

**Storage**
- Data storage
- Backup storage

**Region**
- Pricing varies by region

Pricing Models:
- Pay-as-you-go
- Reserved capacity

# 04 - Cosmos DB

08 April 2026 10:29 PM

What is Azure Cosmos DB?

- **Definition:** Azure Cosmos DB is Microsoft's globally distributed, multi-model database service.
- **Purpose:** It is designed to provide **high availability, low latency, and elastic scalability** for modern applications.
- **Meaning in practice:** It allows developers to store and access data across multiple regions worldwide, with guaranteed performance and consistency.

Cosmos DB stores data like **JSON documents**.

```
{
  "id": "1",
  "name": "John",
  "city": "Paris",
  "role": "Developer"
}
```

Traditional Databases: SQL Server, MySQL, Oracle

Problems:
Hard to scale
Slower globally
Manual management

Cosmos DB solves:
Auto scaling
Global access
High speed
No server management

### Cosmos DB Architecture

Application
↓
Azure Cosmos DB
↓
Data stored globally

### Cosmos DB Features

#### 1. Global Distribution

You can deploy database in:

- India
- US

- Europe
- Asia

Users connect to nearest region → faster performance

### 2. High Availability

Azure Cosmos DB provides:

- 99.999% availability
- Automatic failover

If one region fails → another region works

### 3. Auto Scaling

Cosmos DB automatically:

- Increase performance
- Decrease performance

Based on traffic

### 4. Multi-Model Database

Cosmos DB supports multiple APIs:

| API | Description |
|---|---|
| SQL API | JSON queries |
| MongoDB API | MongoDB compatible |
| Cassandra API | Cassandra support |
| Table API | Key-value |
| Gremlin API | Graph database |

### Cosmos DB Structure

```
Cosmos Account
  └── Database
      └── Container
          └── Items
```

### Partition Key (Important Concept)

Partition Key is used to:

- Distribute data
- Improve performance

Example:
Partition Key:

/city
Data distributed by:

- Pune
- Mumbai
- Delhi

### Throughput (RU/s)

Cosmos DB uses:

### RU/s (Request Units per second)

This measures:

- Read operations
- Write operations
- Query operations

Example:

400 RU/s

Consistency Models Explained

| Model | Meaning | Use Case |
|---|---|---|
| **Strong** | Always returns the latest committed write | Financial transactions |
| **Bounded Staleness** | Reads lag behind writes by a defined window | Social media feeds |
| **Session** | Guarantees consistency within a user session | Shopping carts |
| **Consistent Prefix** | Reads never see out-of-order writes | Event logging |
| **Eventual** | No guarantees, but eventually consistent | Analytics, caching |

Using **Azure Cosmos DB** in .NET

1. **Install NuGet package:**
   dotnet add package Microsoft.Azure.Cosmos
2. **Connect to Cosmos DB:**
   ```
   using Microsoft.Azure.Cosmos;

   string connectionString = "<COSMOS_CONNECTION_STRING>";
   CosmosClient client = new CosmosClient(connectionString);

   Database db = await client.CreateDatabaseIfNotExistsAsync("ePizzaDB");
   Container container = await db.CreateContainerIfNotExistsAsync("Orders", "/customerId");
   ```
3. **Insert a document:**
   ```
   var order = new { id = Guid.NewGuid().ToString(), customerId = "C001", total = 250 };
   await container.CreateItemAsync(order, new PartitionKey(order.customerId));
   ```
4. **Query documents:**
   ```
   var query = container.GetItemQueryIterator<dynamic>("SELECT * FROM Orders o WHERE
   o.customerId = 'C001'");
   while (query.HasMoreResults)
   {
   ```

```
    foreach (var item in await query.ReadNextAsync())
    {
        Console.WriteLine(item);
    }
}
```

# 05 - Azure Web App

08 April 2026 10:17 PM

### Azure App Service Components

Azure App Service contains:

1. **Web Apps**
   - Hosting web applications
   - ASP.NET Core apps
   - MVC apps
   - Angular/React apps
2. **API Apps**
   - Hosting REST APIs
   - Microservices
3. **Mobile Apps**
   - Backend for mobile apps
4. **Function Apps**
   - Serverless applications

### What is Azure Web App

Azure Web App (part of Azure App Service) is a fully managed Platform-as-a-Service (PaaS) that lets you build, deploy, and scale web applications using popular frameworks like .NET, Java, Node.js

### In Simple Words

Instead of:

- Buying server
- Installing IIS
- Managing OS
- Handling deployment

Azure Web App does everything for you.

You just:

- Deploy code
- Run application

### What is "Server" in Azure Web App?

A **Server** is a **computer on the internet** that runs your application and responds to user requests.

### Example

When you open a website:

- You type → www.amazon.com
- Request goes to **Server**
- Server sends back → Website page

So:

Your Browser → Server → Response

### In Azure Web App Context

Normally (Traditional Hosting):
You must:

- Buy a server (computer)
- Install Windows/Linux
- Install Internet Information Services (IIS)
- Deploy your app
- Maintain server

This is called **Server Management**

### But in Microsoft Azure Web App

Azure gives you **ready-made servers**
You **don't see**:

- Server OS
- CPU
- RAM
- IIS

Azure manages everything for you.
You just:

- Upload code
- Run application

### Azure Web App Architecture

User Request
↓
Azure Load Balancer
↓
Azure App Service
↓
Web App Instance
↓
Database / Storage

### Features of Azure Web App

#### 1. Auto Scaling

Automatically scale app based on:

- CPU usage
- Memory
- Request count
- Time schedule

Example:

- Increase instances during peak hours
- Decrease during night

#### 2. Auto Healing

Automatically restarts app if:

- App crashes
- High memory usage
- Slow response

### 3. Built-in Load Balancing

- Azure automatically distributes traffic
- No need to configure load balancer

### 4. Continuous Deployment

Deploy from:

- GitHub
- Azure DevOps
- Bitbucket
- Local Git

### 5. Multiple Deployment Slots

Deployment slots include:

- Production
- Staging
- Testing

#### Deployment Slots Example

Deploy new version to **Staging**
Then swap:

Staging → Production
This avoids downtime.

## Azure Web App Pricing Tiers

| Tier | Description |
|---|---|
| Free | Testing only |
| Shared | Low traffic |
| Basic | Small apps |
| Standard | Production apps |
| Premium | High performance |
| Isolated | Enterprise |

## Deployment Methods

### 1. Visual Studio Deployment

Publish → Azure → Deploy

### 2. GitHub Deployment

Push code → Auto deploy

### 3. FTP Deployment

Upload files using FTP

### 4. Azure CLI

Deploy using command line

# 06 - Azure Function App

08 April 2026 10:38 PM

**What is Azure Function App?**
**Azure Functions** is a **serverless compute service** in **Microsoft Azure** that allows you to run **small pieces of code** without managing servers.
**Simple Meaning**
**Azure Function App runs your code only when needed — and you pay only when it runs.**

In Simple Words

Azure Function App is:

Serverless
Event-driven
Auto scaling
Pay per use

**Azure Web App vs Azure Function App**

| Feature | Azure Web App | Azure Function |
|---|---|---|
| Server | Managed | Serverless |
| Always running | Yes | No |
| Trigger based | No | Yes |
| Cost | Higher | Lower |
| Use case | Web apps | Background jobs |

**Example**
Scenario:
When file uploaded to storage → Process file automatically

File Upload → Azure Function → Process File

**When to Use Azure Function App**
Use Azure Functions when:
- Background jobs
- Scheduled tasks
- Event processing
- APIs (small)
- Automation

**Triggers (Very Important)**
Azure Function runs when **triggered**
Common Triggers:

| Trigger | Description |
| --- | --- |
| HTTP Trigger | API call |
| Timer Trigger | Scheduled job |
| Blob Trigger | File upload |
| Queue Trigger | Message queue |
| Event Grid Trigger | Event based |

**Example Triggers**

**1. HTTP Trigger**

User calls API:

https://myfunction.azurewebsites.net/api/getdata

Function runs.

**2. Timer Trigger**

Run every 5 minutes:

Every 5 minutes → Function runs

Example:

- Send email
- Cleanup logs

**3. Blob Trigger**

File upload → Function runs

Example:

Upload file → Azure Storage → Function runs

**Azure Function Architecture**

Trigger
↓
Azure Function
↓
Execute Code
↓
Output

1. **HTTP Trigger Example:**

```
public static class HelloFunction
{
   [FunctionName("HelloFunction")]
   public static IActionResult Run(
      [HttpTrigger(AuthorizationLevel.Function, "get", "post", Route = null)]
HttpRequest req,
      ILogger log)
```

```
    {
        string name = req.Query["name"];
        return new OkObjectResult($"Hello, {name}");
    }
}
```

2. **Service Bus Trigger Example:**

```
public static class OrderProcessor
{
    [FunctionName("OrderProcessor")]
    public static async Task Run(
        [ServiceBusTrigger("orders", Connection = "ASB_CONNECTION")] string
message,
        ILogger log)
    {
        log.LogInformation($"Processing order: {message}");
        // Save to Cosmos DB or perform business logic
    }
}
```

# 07 - Azure Service Bus

08 April 2026 10:54 PM

**Azure Service Bus (ASB)** is a **message broker service** in **Microsoft Azure** that allows **applications to communicate with each other using messages**.

## Simple Meaning

**Azure Service Bus is used to send messages between applications asynchronously.**

# Why Azure Service Bus?

Suppose you have:

- Order Service
- Payment Service
- Email Service

Without Service Bus:

Order → Payment → Email
If Payment fails → everything fails
With Azure Service Bus:

Order → Service Bus → Payment → Email
Even if Payment is down → message waits in queue

# Why Use Azure Service Bus?

Use when:

- Microservices architecture
- Background processing
- Decoupled applications
- Reliable messaging

# Azure Service Bus Components

## 1. Queue

One sender → One receiver

Sender → Queue → Receiver
Example:

- Order processing

## 2. Topic

One sender → Multiple receivers

Sender → Topic → Multiple Subscribers
Example:

- Order placed → Email + Billing + Notification

# Queue vs Topic

| Feature | Queue | Topic |
|---|---|---|
| Consumers | One | Multiple |
| Messaging | Point-to-point | Publish-subscribe |

Use case Simple processing Event broadcasting

# Message Delivery Modes

## 1. Peek-Lock (Default)

- Message locked
- Process message
- Delete after success

## 2. Receive and Delete

- Message deleted immediately
- No retry

# Dead Letter Queue (Important)

If message fails:
It moves to:

Dead Letter Queue
Used for:

- Error handling
- Retry later

◇ Using **Azure Service Bus (ASB)** in .NET

1. **Install NuGet package:**
   dotnet add package Azure.Messaging.ServiceBus
2. **Send a message to a queue:**

```
using Azure.Messaging.ServiceBus;

string connectionString = "<ASB_CONNECTION_STRING>";
string queueName = "orders";

await using var client = new ServiceBusClient(connectionString);
ServiceBusSender sender = client.CreateSender(queueName);

ServiceBusMessage message = new ServiceBusMessage("Order placed: #1234");
await sender.SendMessageAsync(message);
```

3. **Receive messages:**

```
ServiceBusProcessor processor = client.CreateProcessor(queueName);

processor.ProcessMessageAsync += async args =>
{
   string body = args.Message.Body.ToString();
   Console.WriteLine($"Received: {body}");
   await args.CompleteMessageAsync(args.Message);
};

processor.ProcessErrorAsync += async args =>
{
   Console.WriteLine(args.Exception.ToString());
};
```

# 08 - CI/CD

18 April 2026 08:14 PM

# Introduction to Azure Pipelines

Azure Pipelines is a **CI/CD service** from Microsoft under Azure DevOps that automates:

☑ Build
☑ Test
☑ Deploy
☑ Release

This helps teams **automatically deploy applications** without manual effort.

# Why Azure Pipelines?

Without Azure Pipelines:

Developer → Manual Build → Manual Deployment → Production

With Azure Pipelines:

Developer → Commit → Build → Test → Deploy → Production

Benefits:

☑ Faster deployment
☑ Less manual work
☑ Fewer errors
☑ Continuous delivery

# Azure Pipeline Structure

Azure Pipelines has two main components:

## 1. Build Pipeline (CI)

Used for:

- Compile code
- Run tests
- Generate artifacts

Example:

Code → Build → Test → Artifact

## 2. Release Pipeline (CD)

Used for:

- Deploy application
- Manage environments

Example:

Artifact → Staging → Production

# Azure Pipelines Architecture

Developer → Git Repo → Build Pipeline → Release Pipeline → Production

# Azure Pipelines Platform Support

Azure Pipelines supports:

- .NET
- Java
- Node.js
- Python
- Angular
- React

Deploy to:

- Azure Web Apps
- Azure Kubernetes
- Virtual Machines
- Docker containers

# Understanding CI/CD

CI/CD = Continuous Integration + Continuous Delivery

# Continuous Integration (CI)

Continuous Integration means:

- Developers push code frequently
- Automatically build and test

Example:

Git Push → Build → Test → Ready

Benefits:

☑ Early bug detection
☑ Faster development
☑ Better collaboration

# Continuous Delivery (CD)

Continuous Delivery:

- Build ready for deployment
- Manual approval before production

Example:

Build → Staging → Approval → Production

# Continuous Deployment

Continuous Deployment:

- Automatically deploy to production

Example:

Build → Test → Production (Auto)

# Azure Build Pipeline

Azure Build Pipeline automates:

- Code compilation
- Testing
- Packaging

# Types of Builds

## 1. Source Code Build

Build .NET app

Example:

dotnet build

## 2. Container Image Build

Build Docker image
Example:

docker build -t app .

## 3. Other Build Types

- Maven build
- npm build
- Gradle build

# Creating ASP.NET Core Project

Example:

dotnet new webapi
Push to Git repo:

- GitHub
- Azure Repos

# Creating DevOps Project

Steps:

1. Go to Azure DevOps
2. Create Project
3. Connect Repo
4. Create pipeline

# Azure Build Pipeline Example (.NET)

Example YAML:

```
trigger:
- main

pool:
  vmImage: 'windows-latest'

steps:
- task: DotNetCoreCLI@2
  inputs:
    command: 'build'
```

# Azure Build Pipeline with Azure Web App

Flow:

Code → Build → Publish → Deploy → Azure Web App
Works with:
Azure App Service

# Azure Release Pipeline

Azure Release Pipeline manages deployment:

- Staging
- Production

# Release Pipeline Stages

## Stage 1 — Dev

Deploy to Dev

## Stage 2 — Staging

Test application

## Stage 3 — Production

Deploy live
Example:

Dev → Staging → Production

# Staging Environment

Used for:

- Testing
- Validation
- QA

Example:

test.myapp.com

# Production Environment

Live environment
Example:

www.myapp.com

# Complete CI/CD Workflow

Developer Push Code
↓
Build Pipeline
↓
Run Tests
↓
Create Artifact
↓
Release Pipeline
↓
Deploy to Staging
↓
Approval
↓
Deploy to Production